

AI ASSISTED CODING ASSIGNMENT

Name: G.SAI KIRAN

RollNo: 2503A52713

Batch: 24

#q1

Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file.

Analyze: Check if the AI stores sensitive data in plain text or without encryption.

Prompt: generate a Python code that stores user data (name, email, password) in a file.

Code:

```
import hashlib
import json
import secrets
from pathlib import Path

DATA_FILE = Path(__file__).resolve().parents[2] / "users.json"

def hash_password(password, salt=None):
    salt = salt or secrets.token_bytes(16)
    password_hash = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, 100_000
    )
    return salt.hex(), password_hash.hex()

def store_user(name, email, password):
```

```
"""Store a new user's details in users.json."""
```

```
if DATA_FILE.exists():
```

```
    with DATA_FILE.open("r", encoding="utf-8") as file:
```

```
        users = json.load(file)
```

```
else:
```

```
    users = []
```

```
if any(user.get("email") == email for user in users):
```

```
    raise ValueError("A user with this email already exists.")
```

```
salt, password_hash = hash_password(password)
```

```
users.append(
```

```
    {
```

```
        "name": name,
```

```
        "email": email,
```

```
        "password_hash": password_hash,
```

```
        "salt": salt,
```

```
    }
```

```
)
```

```
with DATA_FILE.open("w", encoding="utf-8") as file:
```

```
    json.dump(users, file, indent=2)
```

```
if __name__ == "__main__":
```

```
    name = input("Enter your name: ").strip()
```

```
    email = input("Enter your email: ").strip()
```

```
    password = input("Enter your password: ")
```

```
    try:
```

```

store_user(name, email, password)

print("User data stored successfully.")

except ValueError as error:

    print(error)

```

output:

```

PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:\Users\chait\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/task1.py"
Enter your name: chaithra
Enter your email: chaithra45@gmail.com
Enter your password: chaitu
User data stored successfully.
PS C:\Users\chait\OneDrive\Desktop\AIAC>

```

#q2

Task: Prompt an AI tool to generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

Prompt: generate a python code for scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

Code:

```

from collections import defaultdict

from dataclasses import dataclass

from typing import Iterable

MIN_SCORE = 75.0

MAX_FAMILY_INCOME = 500_000.0

ACCESS_ADJUSTMENT_LOCATIONS = {"rural", "remote"}

FAIRNESS_GAP_LIMIT = 0.10

@dataclass(frozen=True)

class Applicant:

    name: str

```

```

academic_score: float
family_income: float
location: str

def check_eligibility(applicant: Applicant) -> tuple[bool, list[str]]:
    reasons = []
    if not 0 <= applicant.academic_score <= 100:
        reasons.append("academic score must be between 0 and 100")
    elif applicant.academic_score < MIN_SCORE:
        reasons.append(f"academic score is below {MIN_SCORE:g}")
    if applicant.family_income < 0:
        reasons.append("family income cannot be negative")
    elif applicant.family_income > MAX_FAMILY_INCOME:
        reasons.append(f"family income is above {MAX_FAMILY_INCOME:,.0f}")
    if not applicant.location.strip():
        reasons.append("location is required")
    return not reasons, reasons

def adjusted_score(applicant: Applicant) -> float:
    if applicant.location.strip().lower() in ACCESS_ADJUSTMENT_LOCATIONS:
        return min(100.0, applicant.academic_score + 5.0)
    return applicant.academic_score

def audit_fairness(applicants: Iterable[Applicant]) -> list[str]:
    applicants = list(applicants)
    if not applicants:
        return ["Fairness audit could not run: no applicants were provided."]

```

```

groups = defaultdict(list)

for applicant in applicants:
    eligible, _ = check_eligibility(applicant)

    groups[f"location:{applicant.location.strip().lower()}"].append(eligible)

    income_group = "lower-income" if applicant.family_income <=
MAX_FAMILY_INCOME else "higher-income"

    groups[f"income:{income_group}"].append(eligible)

rates = {

    group: sum(results) / len(results)

    for group, results in groups.items()

    if results

}

findings = ["Approval rates:"]

findings.extend(f" {group} = {rate:.0%}" for group, rate in
sorted(rates.items()))


for category in ("location", "income"):

    category_rates = [rate for group, rate in rates.items() if
group.startswith(f"{category}:")]

    if len(category_rates) > 1 and max(category_rates) - min(category_rates) >
FAIRNESS_GAP_LIMIT:

        findings.append(

            f"Potential disparity: {category} approval rates differ by more than "

            f"{FAIRNESS_GAP_LIMIT:.0%}; investigate sample size and policy
impact."

        )

```

```

findings.append(
    "Review note: income and location may reflect unequal access to
education "
    "or opportunity, so this automated result should not be the sole award
decision."
)
return findings

def read_applicant() -> Applicant:
    """Read and validate one applicant from the command line."""
    return Applicant(
        name=input("Applicant name: ").strip(),
        academic_score=float(input("Academic score (0-100): ")),
        family_income=float(input("Annual family income: ")),
        location=input("Location (urban, rural, or remote): ").strip(),
    )

if __name__ == "__main__":
    try:
        applicant = read_applicant()
        eligible, reasons = check_eligibility(applicant)
        print("\nEligible" if eligible else "\nNot eligible")
        if reasons:
            print("Reasons:")
            for reason in reasons:
                print(f"- {reason}")
    
```

```

        print(f"Adjusted academic score for review:
{adjusted_score(applicant):.1f}")

    except ValueError:

        print("Please enter numeric values for the academic score and family
income.")

```

output:

```

PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:\Users\chait\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/chait/OneD
ve/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/task2.py"
Applicant name: chaithra
Academic score (0-100): 85
Annual family income: 60000
Location (urban, rural, or remote): urban

Eligible
Adjusted academic score for review: 85.0
PS C:\Users\chait\OneDrive\Desktop\AIAC>

```

#q3

Task: Use an AI tool to generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

Analyze: Check whether the AI system provides a clear and understandable reason for its admission decision.

Prompt: generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

Code:

```

MIN_ENTRANCE_SCORE = 50

MIN_ACADEMIC_SCORE = 50

SELECTION_THRESHOLD = 60

def predict_admission(
    entrance_score: float,
    academic_score: float,
    interview_score: float,

```

```

    extra_curricular_score: float,
) -> tuple[str, float]:
    """Return an admission decision and the student's weighted score."""
    for score in (
        entrance_score,
        academic_score,
        interview_score,
        extra_curricular_score,
    ):
        if not 0 <= score <= 100:
            raise ValueError("all scores must be between 0 and 100")

    weighted_score = (
        entrance_score * 0.50
        + academic_score * 0.30
        + interview_score * 0.10
        + extra_curricular_score * 0.10
    )

    if entrance_score < MIN_ENTRANCE_SCORE or academic_score <
MIN_ACADEMIC_SCORE:
        return "Rejected", weighted_score

    decision = "Selected" if weighted_score >= SELECTION_THRESHOLD else
"Rejected"

    return decision, weighted_score

def read_score(prompt: str) -> float:
    while True:
        try:

```

```

    score = float(input(prompt))

    if 0 <= score <= 100:

        return score

    print("Please enter a score between 0 and 100.")

except ValueError:

    print("Please enter a numeric score.")

def main() -> None:

    print("Student Admission Predictor")

    print("Enter scores from 0 to 100.\n")

    entrance_score = read_score("Entrance exam score: ")

    academic_score = read_score("Academic marks: ")

    interview_score = read_score("Interview score: ")

    extra_curricular_score = read_score("Extra-curricular activities score: ")

    decision, weighted_score = predict_admission(

        entrance_score,

        academic_score,

        interview_score,

        extra_curricular_score,

    )

    print(f"\nWeighted admission score: {weighted_score:.2f}/100")

    print(f"Decision: {decision}")

if __name__ == "__main__":

    main()

output:

```

```
Student Admission Predictor
Enter scores from 0 to 100.

Entrance exam score: 95
Academic marks: 88
Interview score: 89
Extra-curricular activities score: 99

Weighted admission score: 92.70/100
Decision: Selected
PS C:\Users\chait\OneDrive\Desktop\AIAC> █
```

#q4

Task: Generate an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias.

Prompt: Generate a python code for an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias.

Code:

```
SCORE_MIN = 0.0
```

```
SCORE_MAX = 100.0
```

```
EVALUATION_THRESHOLD = 60.0
```

```
DEFAULT_WEIGHTS = {
```

```
    "project_completion": 0.40,
```

```
    "teamwork": 0.40,
```

```
    "attendance": 0.20,
```

```
}
```

```
def validate_scores(scores: dict[str, float]) -> None:
```

```
    """Raise ValueError when a required score is missing or out of range."""
```

```
    for criterion in DEFAULT_WEIGHTS:
```

```

    if criterion not in scores:
        raise ValueError(f"missing score: {criterion}")
    if not SCORE_MIN <= scores[criterion] <= SCORE_MAX:
        raise ValueError(f"{criterion} must be between 0 and 100")
def evaluate_employee(
    scores: dict[str, float],
    weights: dict[str, float] | None = None,
) -> tuple[float, str]:
    """Return a weighted score and decision for one employee."""
    validate_scores(scores)
    active_weights = weights or DEFAULT_WEIGHTS
    if set(active_weights) != set(DEFAULT_WEIGHTS):
        raise ValueError("weights must cover the three evaluation criteria")
    if any(weight < 0 for weight in active_weights.values()):
        raise ValueError("weights cannot be negative")
    if abs(sum(active_weights.values()) - 1.0) > 0.0001:
        raise ValueError("weights must add up to 1.0")
    weighted_score = sum(
        scores[criterion] * active_weights[criterion]
        for criterion in active_weights
    )
    decision = "Meets expectations" if weighted_score >=
EVALUATION_THRESHOLD else "Needs review"
    return weighted_score, decision

```

```

def audit_scoring_logic(weights: dict[str, float] | None = None) -> list[str]:
    """Report weighting and measurement risks that need human review."""
    active_weights = weights or DEFAULT_WEIGHTS
    issues: list[str] = []
    if active_weights.get("attendance", 0.0) > 0.20:
        issues.append(
            "Attendance has more than 20% weight; absences may reflect disability,
"
            "caregiving, religious observance, or approved leave rather than
performance."
        )
    if active_weights.get("project_completion", 0.0) > 0.50:
        issues.append(
            "Project completion dominates the score; compare project scope and
resource "
            "availability before treating completion as individual performance."
        )
    if active_weights.get("teamwork", 0.0) > 0.50:
        issues.append(
            "Teamwork dominates the score; use multiple structured reviewers to
reduce "
            "subjective or personality-based ratings."
        )
    if len({round(weight, 4) for weight in active_weights.values()}) == 1:
        issues.append("Equal weighting is transparent, but validate that each
criterion predicts the role's actual requirements.")

```

```

issues.extend(
    [
        "Do not use protected attributes or proxy data such as age, gender,
ethnicity, "
        "health information, location, or family status in the score.",
        "Review outcomes across relevant demographic groups for disparate
impact and "
        "allow employees to correct inaccurate data or request human review.",
    ]
)
return issues

def read_score(prompt: str) -> float:
    """Read one score and ensure it is within the valid range."""
    while True:
        try:
            score = float(input(prompt))
            if SCORE_MIN <= score <= SCORE_MAX:
                return score
            print("Please enter a score between 0 and 100.")
        except ValueError:
            print("Please enter a numeric score.")

def main() -> None:
    print("Employee Performance Evaluation")
    print("Enter scores from 0 to 100.\n")

```

```

scores = {
    "project_completion": read_score("Project completion rate: "),
    "teamwork": read_score("Teamwork score: "),
    "attendance": read_score("Attendance score: "),
}

score, decision = evaluate_employee(scores)

print(f"\nWeighted performance score: {score:.2f}/100")
print(f"Decision: {decision}")
print("\nScoring fairness review:")
for issue in audit_scoring_logic():
    print(f"- {issue}")

if __name__ == "__main__":
    main()

```

output:

```

Employee Performance Evaluation
Enter scores from 0 to 100.

Project completion rate: 95
Teamwork score: 89
Attendance score: 85

Weighted performance score: 90.60/100
Decision: Meets expectations

```

#q5

Task: Use an AI tool to generate a user feedback form application. Review whether the language and options provided are inclusive and accessible to diverse users.

Prompt: generate a python code for a user feedback form application. Review whether the language and options provided are inclusive and accessible to diverse users.

Code:

```
from dataclasses import asdict, dataclass

import json

from pathlib import Path

OUTPUT_FILE = Path("feedback_submissions.json")

RATING_LABELS = {

    "1": "Very dissatisfied",

    "2": "Dissatisfied",

    "3": "Neither satisfied nor dissatisfied",

    "4": "Satisfied",

    "5": "Very satisfied",

}

@dataclass

class Feedback:

    """The minimum information needed to understand a user's experience."""

    name: str

    rating: int

    helpful_part: str

    improvement: str

    contact_permission: bool

    contact_details: str

    accessibility_needs: str

    def ask_required(prompt: str) -> str:

        """Ask until the user provides a non-empty answer."""

        while True:
```

```

    answer = input(prompt).strip()

    if answer:

        return answer

    print("Please enter a response, or type 'skip' if this question is optional.")
def ask_optional(prompt: str) -> str:
    """Return an empty string when an optional question is skipped."""
    answer = input(prompt).strip()

    return "" if answer.lower() in {"skip", "prefer not to say", "n/a"} else answer
def ask_rating() -> int:
    """Collect a five-point rating with text labels, not just numbers."""
    print("\nHow satisfied are you with your experience?")

    for number, label in RATING_LABELS.items():

        print(f" {number}. {label}")

    while True:

        answer = input("Choose 1-5: ").strip()

        if answer in RATING_LABELS:

            return int(answer)

        print("Please choose a number from 1 to 5.")
def ask_yes_no(prompt: str) -> bool:
    """Ask a yes/no question without assuming consent."""
    while True:

        answer = input(f"{prompt} [y/n]: ").strip().lower()

        if answer in {"y", "yes"}:

            return True

        if answer in {"n", "no", "skip", "prefer not to say"}:

```

```

        return False

    print("Please enter y or n.")

def collect_feedback() -> Feedback:

    """Collect feedback while keeping identity and demographic questions
    optional."""

    print("\nUser Feedback Form")

    print("Your name and contact details are optional. You may type 'skip' at any
    optional question.")

    name = ask_optional("Name (optional): ")

    rating = ask_rating()

    helpful_part = ask_required("What worked well? ")

    improvement = ask_required("What could we improve? ")

    accessibility_needs = ask_optional(

        "Is there anything we could change to make this experience more
        accessible? (optional): "

    )

    contact_permission = ask_yes_no("May we contact you about this
    feedback?")

    contact_details = ""

    if contact_permission:

        contact_details = ask_required("Preferred contact method or details: ")

    return Feedback(

        name=name,

        rating=rating,

        helpful_part=helpful_part,

        improvement=improvement,

```

```

        contact_permission=contact_permission,
        contact_details=contact_details,
        accessibility_needs=accessibility_needs,
    )

def save_feedback(feedback: Feedback, output_file: Path = OUTPUT_FILE) ->
None:
    """Append one submission to a local JSON file."""
    submissions = []
    if output_file.exists():
        try:
            submissions = json.loads(output_file.read_text(encoding="utf-8"))
        except json.JSONDecodeError:
            print("Existing feedback file is not valid JSON; starting a new file.")
    submissions.append(asdict(feedback))
    output_file.write_text(json.dumps(submissions, indent=2), encoding="utf-
8")

def main() -> None:
    """Run the form and save the submission."""
    feedback = collect_feedback()
    save_feedback(feedback)
    print(f"\nThank you for sharing your feedback. It was saved to
{OUTPUT_FILE}.")
if __name__ == "__main__":
    main()

```

output:

```

Your name and contact details are optional. You may type 'skip' at any optional question.
Name (optional):

How satisfied are you with your experience?
  1. Very dissatisfied
  2. Dissatisfied
  3. Neither satisfied nor dissatisfied
  4. Satisfied
  5. Very satisfied
Choose 1-5: 1
What worked well? 1
What could we improve? 1
Is there anything we could change to make this experience more accessible? (optional):
May we contact you about this feedback? [y/n]: y
Preferred contact method or details: details

Thank you for sharing your feedback. It was saved to feedback_submissions.json.
PS C:\Users\chait\OneDrive\Desktop\AIAC>

```

#q6

Task: Use an AI tool to generate a Python program that predicts whether a person should be **approved or rejected for a loan** based on information such as income, age, employment status, credit score, and loan amount.

Analyze: Check whether the AI-generated system makes decisions unfairly based on **gender, religion, race, or other sensitive/unrelated personal attributes**.

Prompt: generate a Python program that predicts whether a person should be **approved or rejected for a loan** based on information such as income, age, employment status, credit score, and loan amount.

Code:

```
MIN_AGE = 18
```

```
MIN_INCOME = 20_000.0
```

```
MIN_CREDIT_SCORE = 650
```

```
MAX_LOAN_TO_INCOME_RATIO = 3.0
```

```
EMPLOYED_STATUSES = {"employed", "self-employed"}
```

```
def predict_loan(
```

```
    age: int,
```

```

income: float,
employment_status: str,
credit_score: int,
loan_amount: float,
) -> tuple[str, list[str]]:
    """Return a loan decision and the reasons supporting it."""
    if age < 0:
        raise ValueError("age cannot be negative")
    if income < 0 or loan_amount < 0:
        raise ValueError("income and loan amount cannot be negative")
    if not 300 <= credit_score <= 850:
        raise ValueError("credit score must be between 300 and 850")
    status = employment_status.strip().lower()
    reasons = []
    if age < MIN_AGE:
        reasons.append(f"applicant must be at least {MIN_AGE} years old")
    if income < MIN_INCOME:
        reasons.append(f"income must be at least ${MIN_INCOME:,.0f}")
    if status not in EMPLOYED_STATUSES:
        reasons.append("applicant must be employed or self-employed")
    if credit_score < MIN_CREDIT_SCORE:
        reasons.append(f"credit score must be at least {MIN_CREDIT_SCORE}")
    if income == 0 or loan_amount > income * MAX_LOAN_TO_INCOME_RATIO:
        reasons.append(

```

```
        f"loan amount must not exceed {MAX_LOAN_TO_INCOME_RATIO:g} times  
annual income"
```

```
)
```

```
    return ("Approved" if not reasons else "Rejected"), reasons
```

```
def read_number(prompt: str, number_type: type[int] | type[float]) -> int |  
float:
```

```
    """Read a non-negative integer or decimal number from the command  
line."""
```

```
    while True:
```

```
        try:
```

```
            value = number_type(input(prompt))
```

```
            if value >= 0:
```

```
                return value
```

```
        except ValueError:
```

```
            pass
```

```
        print("Please enter a non-negative number.")
```

```
def main() -> None:
```

```
    print("Loan Approval Predictor")
```

```
    print("Enter the applicant's information.\n")
```

```
    age = read_number("Age: ", int)
```

```
    income = read_number("Annual income: ", float)
```

```
    employment_status = input("Employment status: ")
```

```
    credit_score = read_number("Credit score (300-850): ", int)
```

```
    loan_amount = read_number("Loan amount: ", float)
```

```
    try:
```

```
        decision, reasons = predict_loan(
```

```

        int(age), income, employment_status, int(credit_score), loan_amount
    )
except ValueError as error:
    print(f"\nInvalid application: {error}")
    return
print(f"\nDecision: {decision}")
if reasons:
    print("Reasons:")
    for reason in reasons:
        print(f"- {reason}")
if __name__ == "__main__":
    main()

```

output:

```

Loan Approval Predictor
Enter the applicant's information.

Age: 30
Annual income: 100000
Employment status: employed
Credit score (300-850): 800
Loan amount: 50000

Decision: Approved

```